pyhydra: an open, modular Python library for reproducible hydroclimatic modelling and probabilistic flood-risk analysis

Salvador Navas[a,*], Manuel del Jesus[b]

[a] *IHCantabria – Instituto de Hidráulica Ambiental de la Universidad de Cantabria, Calle Isabel Torres 15, PCTCAN, Santander, 39011, Cantabria, Spain*
[b] *Departamento de Ciencias y Técnicas del Agua y del Medio Ambiente, Universidad de Cantabria, Avenida de los Castros 44, Santander, 39005, Cantabria, Spain*

Abstract

Flood-risk studies typically chain data acquisition, extreme-value and dependence statistics, stochastic scenario generation, climate bias correction and hydrological/hydraulic simulation through ad-hoc, difficult-to-reuse scripts, which hampers reproducibility and transfer between catchments, teams and regulatory contexts. **pyhydra** is an open, modular Python library that consolidates this chain into a single installable scientific core, organised in fifteen sub-modules: harmonised access to hydro-climatic and hydrological data from public services, extreme-value and copula-based statistics, stochastic rainfall generation, climate bias correction, hybrid statistical–dynamical downscaling, automation of HEC-HMS, HEC-RAS, SFINCS and SWAT+, propagation of uncertainty through Monte Carlo ensembles, bootstrap confidence intervals and Bayesian inference, and a pilot-based rule that decides, within a fixed simulation budget, whether an expensive solver's response can be emulated or should instead be sampled by direct Monte Carlo. Individual modules can be used independently once the core package is installed via `pip` from its GitHub repository (not yet on PyPI); a companion platform, HYDRA, optionally exposes the same notebooks through a browser-based Docker deployment, without changing how pyhydra itself is used. pyhydra is illustrated through four flood-risk workflows at three Spanish pilot sites – Besaya, M-30 Manzanares and Valencia – spanning fluvial, urban and compound-event settings, three of which are conditioned on external engines

*Corresponding author.
Email address: `salvador.navas@alumnos.unican.es` (Salvador Navas)
URL: `https://orcid.org/0000-0001-7439-1743` (Salvador Navas), `https://orcid.org/0000-0003-0703-8960` (Manuel del Jesus)

or project data; at Madrid's M-30, the multivariate copula-based methodology it implements yields a 500-year design depth 0.83 m higher than the classical univariate estimate. pyhydra is released under the MIT license, archived on Zenodo, and its methodology already underpins six peer-reviewed journal articles (five published, one in press) from the authors' group.

Keywords: flood risk, reproducible research, hydroclimatic data harmonisation, stochastic rainfall generation, extreme value statistics, uncertainty quantification

---

Required Metadata

Current code version

Nr.	Code metadata description	Metadata
C1	Current code version	v0.2.0
C2	Permanent link to code/repository used for this code version	`https://github.com/navass11/pyhydra/tree/v0.2.0`; archived at `https://doi.org/10.5281/zenodo.21790226`
C3	Legal code license	MIT License
C4	Code versioning system used	git
C5	Software code languages, tools and services used	Python ≥3.9; continuously tested on Python 3.9–3.12 (section 2.2)
C6	Compilation requirements, operating environments and dependencies	Not yet published on PyPI; install with `pip` directly from the GitHub repository in C2, optionally pinned to the tag in C1 (section 2.1). Core dependencies are declared in `pyproject.toml`; optional functionality is distributed through the `statistics`, `geospatial`, `models` and `all` extras (section 2.1). External engines (HEC-HMS, HEC-RAS, SFINCS, SWAT+) are not bundled and must be installed on the host
C7	Link to developer documentation/manual	`https://github.com/navass11/pyhydra/tree/v0.2.0` (README, `docs/` package/installation guide, and `notebooks/pilot_cases/`, which include the four illustrative examples of section 3, inspectable and executable from this repository without HYDRA, subject to the external-engine and project-data requirements listed in Table 3). The companion HYDRA platform additionally exposes the same notebooks through a browser at `https://github.com/navass11/HYDRA/tree/v0.1.2`
C8	Support email for questions	salvador.navas@alumnos.unican.es

Table 1: Code metadata (mandatory).

1. Motivation and significance

Flood-risk assessment usually requires combining several disjoint stages: acquiring precipitation, discharge, soil and climate-projection data from heterogeneous public services; fitting extreme-value and dependence models to characterise the probabilistic structure of hazard; generating or selecting a tractable set of representative scenarios; correcting the bias of climate-model output; and finally running one or more hydrological and hydraulic engines to translate scenarios into physical impacts such as water depth, velocity or inundated area. In practice this chain is implemented as one-off scripts tied to a specific study, which makes results difficult to reproduce, audit or transfer to a different catchment, team or regulatory context – a recurring obstacle in flood-risk management, where under-resourced regions are disproportionately exposed to under-characterised hazard [1].

pyhydra packages a line of methodological work on impact-based flood-risk analysis into reusable software instead of study-specific code. The approach originates in a stochastic, continuous-simulation methodology for the Besaya River at Los Corrales de Buelna (Cantabria), which combined geostatistical event generation with probabilistic regression to derive return periods directly on hydraulic impacts rather than on design hyetographs [2]. Subsequent published work extended this line to spatial stochastic rainfall fields through the companion NEOPRENE library [3], a Gaussian-copula-plus-MaxDiss pipeline for urban flood-frequency analysis in Madrid [4], climate-change impact assessment on reservoir inflows [5] and Andean hydropower systems [6], and vine-copula-based multivariate design storms [7]. Each of these studies re-implemented overlapping pieces of the same statistical and modelling machinery. pyhydra consolidates that machinery into a single, tested, versioned codebase, so that a new case study is assembled from existing building blocks – data connectors, extreme-value fitting, stochastic generators, downscaling operators and model adapters – instead of being rewritten. pyhydra does not replace the hydrological/hydraulic engines it automates (HEC-HMS [8], HEC-RAS [9], SFINCS [10], SWAT+ [11]) or the public data services it connects to [12–15]; it is the reproducible integration layer that turns their combination into an auditable, end-to-end workflow. A companion platform, HYDRA, distributes that layer with a Docker environment and browser interface for transfer to non-specialist users, without being itself part of the code metadata in table 1.

Concretely, pyhydra addresses five recurring needs from the studies above: an installable, dependency-light scientific core; a modular, reproducible architecture usable axis-by-axis; harmonised access to hydro-climatic and hydrological data; automation of hydrological/hydraulic model execution; and

propagation of uncertainty across the whole chain, from Monte Carlo ensembles to Bayesian posteriors – described in section 2 and demonstrated in sections 3 and 4. The companion HYDRA platform separately exposes this core through a web/API/Jupyter/Docker environment (section 2.1).
The contribution is therefore not a new probability distribution or physical solver. It is the software architecture that makes these existing methods composable across the complete path from source data to impact-based frequency products, while retaining the native engines as independent, replaceable components. Statistical components can be used without a hydraulic model, and model adapters can be exchanged without rewriting the climate and uncertainty layers.
The closest open frameworks cover complementary but narrower parts of this path. CLIMADA targets global climate-risk assessment through hazard, exposure and impact functions [16]; RainyDay generates stochastic storms by transposing remotely sensed fields [17]; and HydroMT with wflow automates reproducible construction and execution of distributed hydrological models from global data [18, 19]. pyhydra does not seek to replace those tools. Its distinct scope is to combine multivariate extremes, stochastic generation, climate correction, professional hydrological/hydraulic adapters and impact-based return-period analysis in one inspectable workflow. Where appropriate, it consumes ecosystem tools such as HydroMT-SFINCS as dependencies rather than duplicating them.

2. Software description

2.1. Software architecture

`pyhydra` is a pip-installable Python package with no web or infrastructure code, organised in four functional blocks – data sources, climate and statistics, model integration, and a UQ (uncertainty quantification) strategy layer – that communicate through standard data structures such as `pandas.DataFrame` for tabular series, `xarray.Dataset` for gridded data and GeoTIFF or `geopandas.GeoDataFrame` for spatial layers, so that a module in one block can be replaced or bypassed without breaking the others.
A separate, companion repository, `HYDRA`, consumes pyhydra as a dependency and adds an optional deployment layer on top of it: a JupyterLab environment, a FastAPI backend and an Astro/nginx web frontend, together with the Docker and Azure Container Apps configuration used to deploy them (fig. 1). This layer is what runs the browser-accessible illustrative examples in section 3; pyhydra itself has no dependency on it and can be used directly from any Python environment.

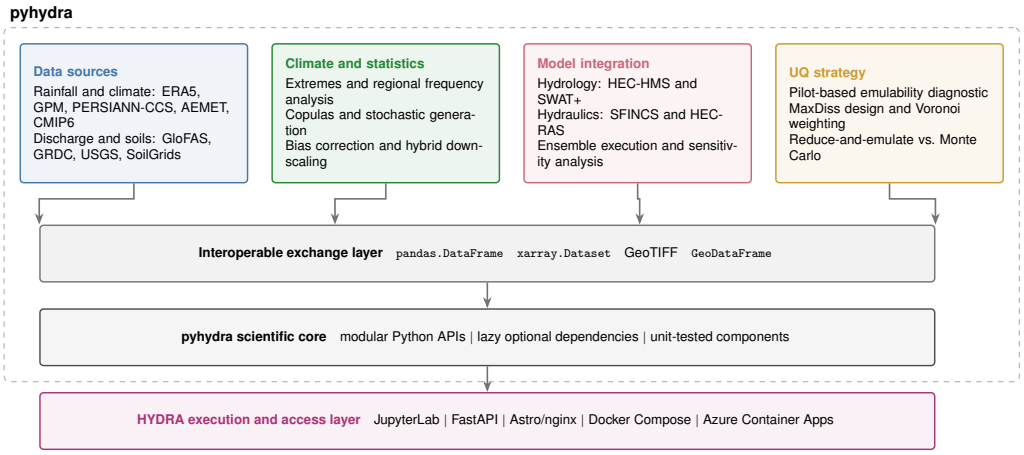

Figure 1: Software architecture. pyhydra provides four functional blocks of reusable Python components connected through standard exchange formats; HYDRA wraps the scientific core with reproducible execution, service and access layers. Colours identify functional roles and do not encode values. ChatGPT and Codex (OpenAI) assisted in writing the TikZ/Python code for this diagram; the authors independently verified the scientific content, edited the source and rendered the final figure.

Each pyhydra sub-module exposes its public API through its `__init__.py`, so the import pattern is always `from pyhydra.<block>.<module> import <name>`. Optional heavy dependencies (PyMC, GDAL, GeoPandas, pyvinecopulib) are imported lazily inside the functions that need them, so the package loads – and the parts of the API that do not need them run – even when those dependencies are absent; a user who only needs extreme-value analysis or bias correction can install pyhydra with `pip` alone. Everything else is grouped into three installable extras – `statistics` (PyMC, pyvinecopulib, lmoments3), `geospatial` (GeoPandas, rasterio, pykrige, earthaccess, hydromt_sfincs) and `models` (hecdss, pySWATPlus, spotpy, NEOPRENE), plus a legacy `geo` extra for bare GDAL bindings and an `all` extra covering all four – so `pip install "pyhydra[statistics]"` installs exactly what a given workflow needs. One extended dependency, CoSMoS_py (spatial random fields), is not yet published on PyPI and must be installed separately from its own repository. HYDRA's Docker Compose stack adds the extended scientific dependencies and orchestrates a JupyterLab container, a FastAPI backend and an nginx-served Astro frontend, built and deployed to Azure Container Apps by a GitHub Actions workflow, with project data mounted from a persistent file share rather than baked into the images.

2.2. Software functionalities

table 2 lists the fifteen sub-modules and their main capabilities. The statistical modules implement established methodology rather than novel formulations: bias correction follows standard delta/quantile-mapping approaches [20], regional frequency analysis follows the index-flood/L-moments framework [21], and dependence modelling follows standard copula theory [22] – consistent with the framing in section 1 that the contribution is architectural, not a new statistical method.

Table 2: pyhydra modules by functional block.

Module	Block	Main capability
`data_sources.rainfall`	Data	ERA5, GPM, PERSIANN-CCS, AEMET, OGIMET, Meteostat
`data_sources.river_discharge`	Data	GloFAS, GRDC, USGS
`data_sources.climate_change`	Data	CMIP6 via CDS and ESGF
`data_sources.soils`	Data	SoilGrids download & USDA texture classification
`climate.time_series`	Climate	Event extraction, GEV/GPD fitting, NSE/KGE/PBIAS
`climate.spatial_analysis`	Climate	Regional frequency analysis, Gaussian/vine copulas, IDW/kriging/GP interpolation
`climate.stochastic_generation`	Climate	NEOPRENE (point NSRP, spatial STNSRP), CoSMoS random fields
`climate.bias_correction`	Climate	Delta method, empirical/quantile-delta/scaled-distribution mapping
`climate.hybrid_downscaling`	Climate	Hydrograph classification, MaxDiss selection, k-NN/RBF flood-map reconstruction
`modeling.hydrology.hec_hms`	Modelling	Project generation, DSS I/O, SCE-UA calibration via spotpy
`modeling.hydrology.swat`	Modelling	SWAT+ input generation, execution, output parsing
`modeling.hydraulic.sfincs`	Modelling	Model setup, boundary conditions, batch execution
`modeling.hydraulic.hec_ras`	Modelling	1D/2D project automation, DSS result extraction
`modeling.hydraulic.sensitivity`	Modelling	Manning Monte Carlo ensembles, spatial impact regression
`uq`	UQ	Pilot-based emulability diagnostic, MaxDiss design, reduce-and-emulate vs. Monte Carlo

Abbreviations: GEV/GPD, Generalised Extreme Value / Generalised Pareto Distribution; NSE/KGE/PBIAS, Nash–Sutcliffe Efficiency / Kling–Gupta Efficiency / Percent Bias; IDW, Inverse Distance Weighting; GP, Gaussian Process; DSS, Data Storage System (HEC file format); SCE-UA, Shuffled Complex Evolution – University of Arizona; RBF, Radial Basis Function.

Four design principles guide the implementation: *modularity* (each block can be used independently – extreme-value analysis does not require geospatial or modelling dependencies); *reproducibility* (standardised notebooks, explicit parameters and data conventions preserve the evidence needed to regenerate a result, while identifying any required credentials, input projects or host-installed engines); *interoperability* (NetCDF/`xarray`, GeoTIFF, `pandas` and GeoJSON/`geopandas` as the common data currency between blocks and with external tools such as QGIS); and *operational transfer* (documentation, example notebooks and a reproducible Docker environment are treated as first-class deliverables, not optional accessories). A `pytest` suite of 291 tests across seventeen modules exercises the statistical, modelling and UQ utilities most exposed to regression. A GitHub Actions workflow (Ubuntu, core plus lightweight extended dependencies including PyMC) runs this suite on every push across a Python 3.9–3.12 matrix; the one test that depends on PyMC/pytensor is still marked `optional_dependency` and guarded to skip rather than fail if that stack is unavailable or, as happens with the older PyMC resolved on Python 3.9, incompatible with the installed NumPy. For the code version in Table 1 (commit `5552aa9`, 4 August 2026), this workflow reports 291/291 passed on Python 3.10–3.12 and 290 passed plus 1 skipped on 3.9, with 30–31% line coverage – concentrated in the statistical and modelling-adapter code exercised directly by unit tests, lower in the data-connector and engine-automation modules whose behaviour is instead exercised, not formally verified, by the pilot-case notebooks of section 3 against real services and engines. Runs archived at `https://github.com/navass11/pyhydra/actions/runs/30904958268`.

Reproducibility boundary. Reproducibility is reported at the level supported by each workflow. Extreme-value fitting, copula analysis and bias-correction examples are executable from included or synthetic data in the container. Public-data downloads are repeatable after the user supplies the corresponding service credentials and records the query metadata. HEC-HMS, HEC-RAS, SFINCS and SWAT+ workflows additionally require the relevant engine and a model project; large pilot cases may also contain third-party data that cannot be redistributed. HYDRA preserves notebooks, parameters, adapters and standard outputs for these conditioned workflows, but does not claim that containerising the Python layer redistributes proprietary executables or restricted datasets.

Uncertainty propagation and analysis. Rather than being confined to a single module, uncertainty quantification cuts across the four functional blocks: asymptotic, bootstrap and Bayesian posterior estimates on extreme-

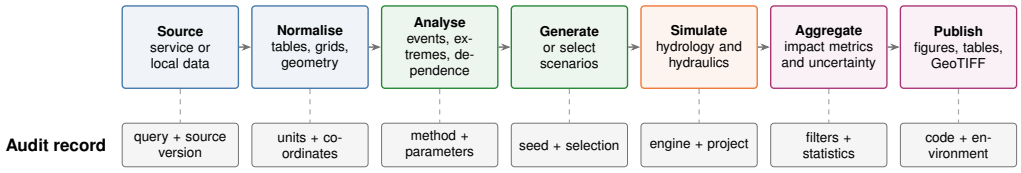

Figure 2: Reproducible workflow contract. Each transformation produces a scientific artefact and an audit record sufficient to identify its source, parameters, stochastic seed, external-engine configuration and software environment. Colours identify functional stages and do not encode values. ChatGPT and Codex (OpenAI) assisted in writing the TikZ/Python code for this diagram; the authors independently verified the scientific content, edited the source and rendered the final figure.

value parameters and return levels (`climate.time_series`); posterior propagation across pooled stations and prediction variance from spatial interpolators (`climate.spatial_analysis`); and Monte Carlo roughness ensembles aggregated into spatial impact statistics with outlier screening (`modeling.hydraulic`) – the machinery exercised by the SFINCS–HEC-RAS demonstration case in section 3. The `uq` module is a dedicated decision layer sitting above these: given an ensemble and an expensive solver, it draws a nested maximum-dissimilarity pilot charged against a fixed evaluation budget `n_max`, measures that pilot's cross-validated emulability against a declared family of cheap emulators, and from that measurement chooses between completing a reduce-and-emulate design or falling back to direct Monte Carlo – rather than fixing the choice by convention. A `StrategyDecision.margin` flags choices taken on a hair's breadth, and a companion sequential-design routine classifies each additional design point as improving, converged or deteriorating so that a flattened error curve is not mistaken for one that has turned upward.

2.3. Sample code snippets

The snippet below chains three independent pyhydra components – bias correction, stochastic rainfall generation and bivariate copula analysis – each usable on its own:

```
from pyhydra.climate.bias_correction import BiasCorrection
from pyhydra.climate.stochastic_generation.point import NSRPModel
from pyhydra.climate.spatial_analysis.copulas import BivariateCopula

# 1. Bias-correct a regional climate model precipitation series
corrected = BiasCorrection.fit_transform(
    obs, hist, future, var='pr', method='qdm')

# 2. Calibrate and simulate a Neyman-Scott rectangular-pulse
```

```
# stochastic rainfall generator (NEOPRENE)
model = NSRPModel(seasonality='monthly')
model.fit(observed_series)
synthetic = model.simulate(year_ini=2000, year_fin=2100)

# 3. Joint return period of a compound rainfall-discharge event
copula = BivariateCopula(family='gumbel').fit(rain_max, discharge_max)
x_or, y_or = copula.joint_return_period(t=100, kind='or')
```

3. Illustrative examples

pyhydra has been exercised through four flood-risk workflows at three Spanish pilot sites, each reproduced as a notebook sequence under `notebooks/pilot_cases/` in the pyhydra repository itself (table 1, C7); the companion HYDRA platform additionally exposes the same notebooks through a browser, without requiring the underlying pyhydra sequence to change. table 3 traces each workflow to its permanent notebook sequence and states, per the reproducibility boundary defined above, what is required to re-run it.

Los Corrales de Buelna (Besaya River, Cantabria). The founding case of the methodology [2]: 42 years of continuous hydrological simulation (1970–2012) driven by 13 rain gauges, combined with geostatistical event generation and k-NN probabilistic regression ($k = 6$) to estimate return periods directly on flood impacts rather than on design-storm discharge, in a valley-bottom town where more than 13% of the population lies within the mapped floodplain.

M-30 Manzanares (Madrid). Reproduces the multivariate flood frequency methodology published by Navas et al. [4] (fig. 3): 1,761 historical events from 17 rain gauges are classified by PCA + k-means, an OpenTURNS Gaussian copula generates approximately one million synthetic events over the resulting parameter space, MaxDiss selects 1,000 representative events, which are simulated in HEC-HMS and steady-state HEC-RAS 1D (303 cross-sections), and a k-NN regressor reconstructs depths across the full synthetic ensemble under a compound Poisson frequency model. The resulting T=500-year design depth at Puente de Toledo exceeds the classical univariate-GEV estimate by 0.83 m (7.61 m vs. 6.78 m), illustrating how neglecting inter-gauge dependence can materially underestimate design hazard in a densely instrumented urban catchment.

Valencia DANA (October 2024). Implements an extreme-value fitting comparison for the record-breaking rainfall event that struck the Comunitat Valenciana on 29 October 2024, when AEMET's official post-event

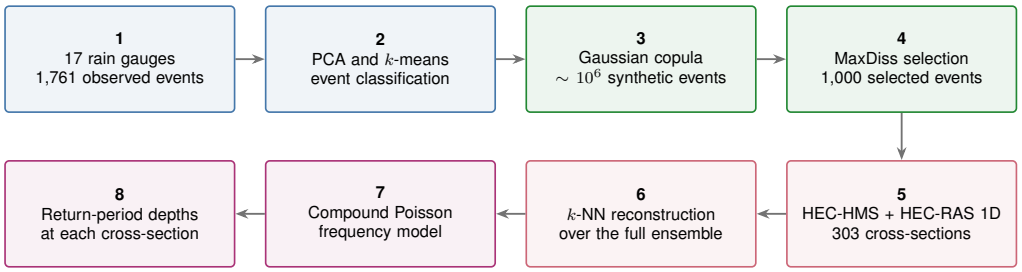

Figure 3: End-to-end pipeline reproduced in the M-30 Manzanares pilot case: from 17 rain-gauge series (step 1) to return-period depths at two control cross-sections, Puente de Toledo and Represa nº9 (step 8), following Navas et al. [4]. Blue denotes observations and event classification, green statistical generation and selection, orange physical simulation and emulation, and purple frequency outputs. ChatGPT and Codex (OpenAI) assisted in writing the TikZ/Python code for this diagram; the authors independently verified the scientific content, edited the source and rendered the final figure.

report [23] recorded new national single-station intensity records at Turís (station 8337X); the dataset used here, aggregated on calendar days, gives 710.8 mm/24h at that station. Two notebooks, downloadable in full with `pyhydra-get-data valencia_dana` and requiring no AEMET credentials at run time, fit GEV distributions at Turís with and without the event: single-station fits by three methods (MLE, L-moments, Bayesian/MCMC), and a pooled-regional fit (index-flood standardisation over all nine stations) by MLE and L-moments. Including the event raises the estimated T=100-year return level by a factor of 3.2–4.6 depending on method and scale, from 218–271 mm to 756–1,241 mm (fig. 4).

SFINCS–HEC-RAS uncertainty comparison (Besaya River). A fourth demonstration case exercises the uncertainty-propagation machinery of section 2.2 across two independent 2D hydraulic engines: `generate_manning_combinations` draws 1,000 Monte Carlo roughness realisations over nine land-use classes from literature distributions, the same ensemble is run through both SFINCS and HEC-RAS, and `load_sfincs_ensemble`/`load_flood_ensemble` together with `spatial_stats` and `manning_flood_regression` aggregate each engine's inundation output into comparable spatial impact statistics (fig. 5). This section presents only the pyhydra ensemble-generation and aggregation workflow, on one engine, as a software demonstration; the resulting two-engine scientific comparison between SFINCS and HEC-RAS is out of scope here and reported separately.

The four workflows follow complementary execution paths – continuous impact-frequency analysis with hydraulic ensembles (Besaya), rainfall, runoff

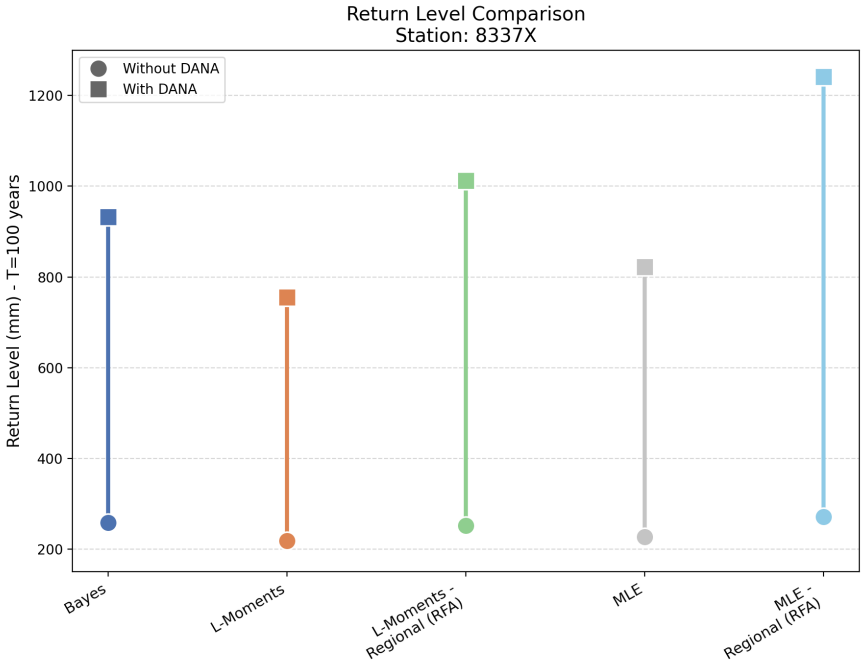

Figure 4: T=100-year return-level estimates at station 8337X (Turís), with and without the 29 October 2024 event, for the single-station (MLE, L-moments, Bayesian/MCMC) and pooled-regional-RFA (MLE, L-moments) fits reproduced by the two archived notebooks described in the text.

and hydraulic modelling (M-30), and statistical inference without an external solver (Valencia) – and are not the only published applications of the underlying methodology: the same building blocks also support published work on Andean hydropower systems [6] and reservoir inflows under climate change [5].

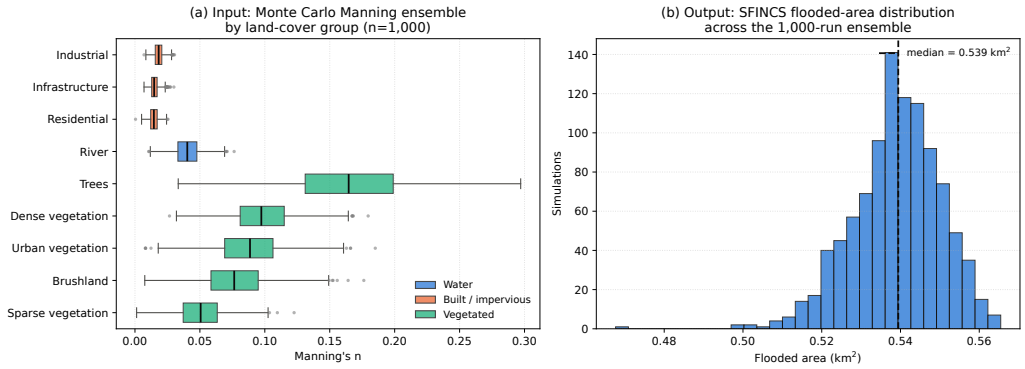

Figure 5: (a) Input ensemble generated by pyhydra's `generate_manning_combinations`: 1,000 Manning-roughness combinations over nine land-use classes, grouped by land-cover type (water, built/impervious, vegetated; boxes: interquartile range; line: median; whiskers: non-outlier range; dots: outliers). (b) Output: flooded-area distribution across the same 1,000-run SFINCS ensemble, computed from the archived simulation outputs with pyhydra's `load_flood_ensemble` and `flooded_area`. This single-engine distribution documents the ensemble-aggregation workflow only; the two-engine SFINCS/HEC-RAS inundation comparison is out of scope here and reported separately.

Table 3: Traceability of the four illustrative workflows. Notebook paths are relative to `https://github.com/navass11/pyhydra/tree/v0.2.0/notebooks/pilot_cases/`. "Open data" marks inputs redistributable in full; "Partial" marks cases where only some inputs (e.g. literature-based parameter distributions or published aggregates) are open and the remainder is project-specific and not redistributed here.

Case	Path	Open data	External engine	Reproduction
Besaya	`los_corrales_buelna/` (9 nb.*)	Partial	HEC-HMS, HEC-RAS	Conditioned on engine + project data
M-30 Manzanares	`m30_manzanares/` (6 nb.)	Partial	HEC-HMS, HEC-RAS	Conditioned on engine + project data
Valencia DANA	`valencia_dana/` (2 nb.)	Yes (AEMET, via `pyhydra-get-data`)	None	Open and reproducible for the single-station and pooled-regional analyses
SFINCS–HEC-RAS	`manning_rugosidades/` (7 nb.)	Partial (Manning distributions open; terrain/mesh project-specific)	SFINCS, HEC-RAS	Conditioned on engine + project data

*Eight sequential steps (01–06, 08); step 07 has two notebook variants (`07_hec_ras_hydraulics.ipynb`, the one referenced by step 06 and used for the results reported here, and an auxiliary `07_hec_ras_hydraulics_run.ipynb`).

4. Impact

pyhydra formalises, as versioned and tested software, a line of flood-risk research that previously existed as independent, non-interoperable study codebases. Five published journal articles and one journal article in press, produced by the authors' group, already rely on methods now consolidated in pyhydra [2–7]. Conference proceedings and manuscripts still under review are not counted here as impact evidence. This methodological track record predates pyhydra as a software product and shows that the underlying methods work; it is distinct from, and does not substitute for, evidence that the software itself has been adopted outside the authors' group.

Beyond consolidating methods that already existed as separate scripts, having them share a common ensemble and impact-statistics layer (section 2.2) also opens questions that were impractical to pose across four disconnected codebases: how differently two hydraulic engines propagate the same roughness ensemble into inundation extent, as exercised in section 3; and how directly conditioning frequency analysis on spatial impacts, rather than on discharge or rainfall alone, shifts design estimates relative to the classical univariate route, as the M-30 comparison in section 3 begins to quantify.

As a software product, pyhydra's impact so far is internal to the authors' group, reported as such rather than overstated. A new case study is now assembled from existing, unit-tested building blocks rather than rewritten – what let the four workflows of section 3 share one code base instead of four disconnected scripts, and what made the M-30 comparison against the classical univariate method (0.83 m higher T=500-year depth) straightforward rather than a bespoke re-implementation. Archiving pyhydra under the MIT license on Zenodo [24] gives each downstream publication a citable, versioned code reference, closing a gap that affected the earlier, unpublished study scripts. The companion HYDRA platform (section 2.1), archived separately [25] and reachable, as of 30 July 2026, at the URL given in its README[1], removes local-environment setup as a barrier to inspecting any of the four notebooks with the full scientific stack pre-installed.

Because the public GitHub release is recent, independent adoption metrics – downloads, external contributions, third-party citations, teaching or commercial use – are not yet available, and none are claimed here. pyhydra is used in ongoing doctoral and project research at IHCantabria – Universidad de Cantabria; broadening use beyond the authors' group is future work (section 5).

[1]Ephemeral Azure Container Apps deployment; useful for evaluation but not a citable scholarly reference. The Zenodo records remain the durable citation for both components.

5. Conclusions

pyhydra turns a decade-long line of impact-based flood-risk research into modular, tested, MIT-licensed software: an installable scientific core with a modular, reproducible architecture; harmonised access to hydro-climatic and hydrological data; automation of hydrological and hydraulic models; and propagation and analysis of uncertainty across the whole chain. The companion HYDRA platform separately provides an optional operational web, API, Jupyter and Docker environment on top of this core, without changing how pyhydra itself is installed or used. Four workflows at three Spanish pilot sites – Besaya, M-30 Manzanares and Valencia – span fluvial, urban and compound-event flood risk. Together they show that the same modular components support markedly different study designs, from continuous stochastic simulation to multivariate copula-based frequency analysis and cross-engine Monte Carlo comparison. Ongoing work focuses on widening automated test coverage as new modules are added, adding adapters for further hydraulic/hydrological engines, and documenting a stable public API to lower the barrier for external contributions beyond the authors' own research group.

The present release also has explicit limits. Short records and rare extremes remain information-limited even under bootstrap, Bayesian or regional uncertainty quantification, and regional analysis still requires a defensible homogeneous region; stochastic generators do not by themselves resolve climate non-stationarity; emulators and MaxDiss selection reduce hydraulic-ensemble cost but cannot replace physical-model validation, and depend on the representativeness of the simulated subset; and reproducibility of externally executed models is conditioned by engine version, licences and project availability. HYDRA makes these dependencies explicit and auditable rather than removing them.

CRediT authorship contribution statement

Salvador Navas: Conceptualization, Methodology, Software, Data curation, Formal analysis, Investigation, Validation, Visualization, Writing – original draft, Writing – review & editing. **Manuel del Jesus:** Conceptualization, Methodology, Resources, Supervision, Funding acquisition, Validation, Writing – review & editing.

Funding

Manuel del Jesus acknowledges the support provided by Project PCI2024-153483 (WaMA-WaDiT, "Water Management and Adaptation based on Wa-

tershed Digital Twins"), funded by MICIU/AEI/10.13039/501100011033 and co-funded by the European Union. This research did not receive any other specific grant from funding agencies in the public, commercial or not-for-profit sectors. The funding bodies had no role in study design, data collection, analysis, interpretation, manuscript preparation or the decision to submit the article for publication.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The pyhydra version described in this article is available through the versioned GitHub and Zenodo records in Table 1. The companion HYDRA deployment platform used to run the browser-accessible workflows in section 3 is likewise open source and MIT-licensed, and is archived separately [25]. The pilot workflows additionally consume public third-party data services cited in the text. Proprietary hydraulic-engine projects and restricted project datasets cannot be redistributed; their absence does not prevent inspection or reuse of the open statistical components, interfaces and example workflows.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During manuscript preparation (2026), the authors used ChatGPT and the Codex coding assistant (OpenAI; specific model versions not recorded) to assist with language editing, text revision, manuscript formatting and the drafting of figure code (identified in the affected captions); during these revisions (30 July – 4 August 2026), Claude Sonnet 5 (model ID `claude-sonnet-5`, Anthropic) assisted with editing manuscript text, diagnosing and fixing software defects surfaced by the continuous-integration workflow added in section 2.2, and re-executing the Valencia notebooks to regenerate fig. 4 from their output. In all cases, assistance was limited to code and text; it did not extend to designing the underlying methodology or validating its scientific content. After using these tools, the authors reviewed and edited the content and code as needed, checked the cited evidence and numerical values, and take full responsibility for the content of the published article and the archived code.

Acknowledgements

The stochastic-impact methodology consolidated in this software originates from doctoral and research work carried out at IHCantabria – Instituto de Hidráulica Ambiental de la Universidad de Cantabria. The authors thank the Confederación Hidrográfica del Cantábrico and the Agencia Estatal de Meteorología for hydrological and climate records used in the associated case studies, and the Instituto Geográfico Nacional for openly available PNOA products. Computationally intensive workflows used IHCantabria infrastructure and resources provided by the University of Cantabria through its Microsoft Azure academic subscription.

References

[1] G. Di Baldassarre, A. Montanari, H. Lins, D. Koutsoyiannis, L. Brandimarte, G. Blöschl, Flood fatalities in Africa: from diagnosis to mitigation, Geophys. Res. Lett. 37 (22) (2010). doi:10.1029/2010GL045467.

[2] S. Navas, M. del Jesus, J. M. Sánchez, Evaluación y análisis del riesgo de inundación del río besaya a su paso por los corrales de buelna, cantabria, Revista de Obras Públicas 3598 (2018) 61–72.

[3] J. Diez-Sierra, S. Navas, M. del Jesus, NEOPRENE v1.0.1: A Python library for generating spatial rainfall based on the neyman-scott process, Geosci. Model Dev. 16 (2023) 5035–5048. doi:10.5194/gmd-16-5035-2023.

[4] S. Navas, M. del Jesus, J. Martín, P. Sánchez, Application of a new methodology to improve the estimation of flood frequencies in calle 30, madrid, Ingeniería del Agua 28 (4) (2024) 263–279. doi:10.4995/ia.2024.22293.

[5] S. Navas, M. del Jesus, SIMPCCe: A tool for the analysis of reservoir inflows under climate change scenarios, Ingeniería del Agua 29 (2) (2025) 132–148. doi:10.4995/ia.2025.23217.

[6] M. del Jesus, J. Paz, S. Navas, E. Turienzo, J. Diez-Sierra, N. Pena, Efectos del cambio climático en el recurso hídrico de los países andinos, Ingeniería del Agua 24 (4) (2020) 219–233. doi:10.4995/ia.2020.12135.

[7] D. Urrea Méndez, D. V. Gómez Rave, M. del Jesus, Multivariate design storms using vine copulas and Gaussian process regression, J. Hydrol.In press (2026). doi:10.1016/j.jhydrol.2026.135762.

[8] W. Scharffenberg, Hydrologic modeling system HEC-HMS: User's manual, Tech. rep., US Army Corps of Engineers, Hydrologic Engineering Center (2016).

[9] G. W. Brunner, HEC-RAS river analysis system: 2d modeling user's manual, Tech. rep., US Army Corps of Engineers, Hydrologic Engineering Center (2016).

[10] T. Leijnse, M. van Ormondt, K. Nederhoff, A. van Dongeren, Modeling compound flooding in coastal systems using a computationally efficient reduced-physics solver: including fluvial, pluvial, tidal, wind- and wave-driven processes, Coast. Eng. 163 (2021) 103796. doi:10.1016/j.coastaleng.2020.103796.

[11] P. W. Gassman, M. R. Reyes, C. H. Green, J. G. Arnold, The soil and water assessment tool: Historical development, applications, and future research directions, Trans. ASABE 50 (4) (2007) 1211–1250. doi:10.13031/2013.23637.

[12] H. Hersbach, B. Bell, P. Berrisford, S. Hirahara, A. Horányi, J. Muñoz-Sabater, J. Nicolas, C. Peubey, R. Radu, D. Schepers, A. Simmons, C. Soci, S. Abdalla, X. Abellan, G. Balsamo, P. Bechtold, G. Biavati, J. Bidlot, M. Bonavita, D. Dee, R. Dragani, J. Flemming, R. Forbes, A. Geer, L. Haimberger, S. Healy, R. J. Hogan, E. Holm, M. Janisková, S. Keeley, P. Laloyaux, P. López, C. Lupu, G. Radnoti, P. de Rosnay, I. Rozum, F. Vamborg, S. Villaume, J.-N. Thépaut, The ERA5 global reanalysis, Q. J. R. Meteorol. Soc. 146 (730) (2020) 1999–2049. doi:10.1002/qj.3803.

[13] G. J. Huffman, D. T. Bolvin, D. Braithwaite, K.-l. Hsu, R. J. Joyce, C. Kidd, E. J. Nelkin, S. Sorooshian, E. F. Stocker, J. Tan, D. B. Wolff, P. Xie, Integrated multi-satellite retrievals for the Global Precipitation Measurement (GPM) mission (IMERG), in: Satellite Precipitation Measurement, Springer, 2020, pp. 343–353. doi:10.1007/978-3-030-24568-9_19.

[14] L. Alfieri, P. Burek, E. Dutra, B. Krzeminski, D. Muraro, J. Thielen, F. Pappenberger, GloFAS – global ensemble streamflow forecasting and flood early warning, Hydrol. Earth Syst. Sci. 17 (2013) 1161–1175. doi:10.5194/hess-17-1161-2013.

[15] L. Poggio, L. M. de Sousa, N. H. Batjes, G. B. M. Heuvelink, B. Kempen, E. Ribeiro, D. Rossiter, SoilGrids 2.0: producing soil information for

the globe with quantified spatial uncertainty, SOIL 7 (2021) 217–240. doi:10.5194/soil-7-217-2021.

[16] G. Aznar-Siguan, D. N. Bresch, Climada v1: a global weather and climate risk assessment platform, Geosci. Model Dev. 12 (2019) 3085–3097. doi:10.5194/gmd-12-3085-2019.

[17] D. B. Wright, R. Mantilla, C. D. Peters-Lidard, A remote sensing-based tool for assessing rainfall-driven hazards, Environ. Model. Softw. 90 (2017) 34–54. doi:10.1016/j.envsoft.2016.12.006.

[18] D. Eilander, et al., Hydromt: Automated and reproducible model building and analysis, J. Open Source Softw. 8 (2023) 4897. doi:10.21105/joss.04897.

[19] W. J. van Verseveld, A. H. Weerts, M. Visser, et al., Wflow_sbm v0.7.3, a spatially distributed hydrological model: from global data to local applications, Geosci. Model Dev. 17 (2024) 3199–3234. doi:10.5194/gmd-17-3199-2024.

[20] C. Teutschbein, J. Seibert, Bias correction of regional climate model simulations for hydrological climate-change impact studies: Review and evaluation of different methods, J. Hydrol. 456–457 (2012) 12–29. doi:10.1016/j.jhydrol.2012.05.052.

[21] J. R. M. Hosking, J. R. Wallis, Regional Frequency Analysis: An Approach Based on L-Moments, Cambridge University Press, 1997. doi:10.1017/CBO9780511529443.

[22] C. Genest, A.-C. Favre, Everything you always wanted to know about copula modeling but were afraid to ask, J. Hydrol. Eng. 12 (4) (2007) 347–368. doi:10.1061/(ASCE)1084-0699(2007)12:4(347).

[23] Agencia Estatal de Meteorología (AEMET), Informe sobre el episodio meteorológico de precipitaciones torrenciales y persistentes ocasionadas por una dana el día 29 de octubre de 2024, Tech. rep., AEMET, Ministerio para la Transición Ecológica y el Reto Demográfico, prepared with information available as of 8 November 2024 (2024).
URL https://www.aemet.es/documentos/es/conocermas/recursos_en_linea/publicaciones_y_estudios/estudios/informe_episodio_dana_29_oct_2024_.pdf

[24] S. Navas Fernández, M. del Jesus, pyhydra: a modular Python library for hydrological and climate analysis, computer software, version 0.2.0

(2026). doi:10.5281/zenodo.21790226.
URL https://doi.org/10.5281/zenodo.21790226

[25] S. Navas Fernández, M. del Jesus, HYDRA: web platform, notebooks and deployment environment for pyhydra, computer software, version 0.1.2 (2026). doi:10.5281/zenodo.21705293.
URL https://doi.org/10.5281/zenodo.21705293

Current executable software version

pyhydra v0.2.0 (commit 5552aa9) is the executable software version described in this publication; its archived record and immutable GitHub tag link are provided in Table 1 and should be cited independently using the corresponding Zenodo record [24]. The companion HYDRA deployment platform used for the browser-accessible examples in section 3 is archived separately at v0.1.2 (commit 627a838) [25].